Runtime IDS Test Guide

STEP 1 - Generate normal traffic

```
python backend/ml/simulator/traffic_generator.py --frames 5000 --attack none --out backend/ml/outputs/replay_logs/normal.csv
```

STEP 2 - Inject fuzzy attack traffic

```
python backend/ml/simulator/traffic_generator.py --frames 5000 --attack fuzzy --intensity 0.8 --out backend/ml/outputs/replay_logs/fuzzy.csv
```

STEP 2B - Generate DoS traffic

```
python backend/ml/simulator/traffic_generator.py --frames 5000 --attack dos --intensity 0.8 --out backend/ml/outputs/replay_logs/dos.csv
```

Purpose: Simulate CAN bus flooding / timing domination attack.

Expected IDS behavior: rapid alert escalation, timing anomalies dominate, burst detection activates, HIGH/CRITICAL alerts possible.

STEP 3 - Replay traffic into realtime IDS

```
python backend/ml/runtime/replay_runner.py --input backend/ml/outputs/replay_logs/normal.csv

python backend/ml/runtime/replay_runner.py --input backend/ml/outputs/replay_logs/fuzzy.csv
```

STEP 3B - Replay DoS traffic into realtime IDS

```
python backend/ml/runtime/replay_runner.py --input backend/ml/outputs/replay_logs/dos.csv
```

Expected runtime validation: alert count much higher than normal, dominant detection should shift to timing, burst/cadence anomalies appear, low detection latency.

STEP 4 - View alerts

```
JSONL: backend/ml/outputs/alerts/alerts.jsonl

CSV:   backend/ml/outputs/alerts/alerts.csv
```

STEP 5 - Benchmark runtime performance

```
python backend/ml/validation/benchmark_runtime.py --input backend/ml/outputs/replay_logs/fuzzy.csv

Output: backend/ml/outputs/validation_reports/runtime_benchmark.json
```

Expected IDS Behavioral Validation

```
| Replay Type | Expected Behavior |

| normal.csv | quiet / near-zero alerts |

| fuzzy.csv | payload anomalies / continuity breaks |

| dos.csv | timing bursts / dominance anomalies |

| rpm.csv | slower sustained hybrid anomalies |

| gear.csv | temporal continuity anomalies |
```